Tecnicatura Superior en Desarrollo de Software

Anexo 07e — Fundamentos de `Blazor`: Componentes, Binding e Inyección de Servicios

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026

¿Qué es un componente Blazor?

Un componente Blazor es un archivo `.razor` que combina marcado HTML con código C# en un mismo archivo. El compilador de Razor transforma ese archivo en una clase C# normal — cada componente *es*, en tiempo de ejecución, una clase que hereda de `ComponentBase`. El marcado HTML se convierte en el método que produce el árbol de renderizado; el bloque `@code { ... }` se convierte literalmente en el cuerpo de esa clase: campos, propiedades y métodos comunes de C#.

Esta dualidad es la clave para entender todo lo demás en este anexo: cuando un componente "recibe un servicio inyectado", no es magia de plantilla — es el mismo mecanismo de Inyección de Dependencias que ya conocés (**Anexo 04c**), aplicado a una clase que además sabe dibujar HTML.

Componente Blazor	Marcado + C# en un mismo archivo, compilado a una clase	.razor

Un archivo `.razor` declara la UI de una pantalla o de una pieza reutilizable de UI. El marcado usa directivas (`@bind`, `@onclick`, `@inject`...) para conectar HTML con el código C# del mismo archivo.

Directivas que cubre este anexo:

- `@bind` — binding bidireccional
- `@onclick` / `EventCallback` — eventos
- `[Parameter]` — datos desde el padre
- `@inject` / `OwningComponentBase<T>` — inyección de servicios

Alcance de este anexo: esto es lo mínimo necesario para entender el ViewModel y la View de MVVM que se ven en el Módulo 07b. Vas a ver `OnInitialized()` — el método de ciclo de vida más simple, para resolver un servicio apenas se crea el componente — pero quedan fuera a propósito el resto del ciclo de vida (`OnInitializedAsync`, `OnParametersSetAsync`, `OnAfterRender`, etc.) y la interoperabilidad con JavaScript — este curso no los necesita.

Data binding con `@bind`

`@bind` conecta un elemento HTML con un campo o propiedad del `@code` en las dos direcciones: si el campo cambia por código, el input se actualiza en pantalla; si el usuario escribe en el input, el campo se actualiza en el objeto C#. Por debajo, el compilador de Razor expande `@bind="titulo"` en un atributo `value` más un manejador del evento `onchange` que reasigna el campo — es azúcar sintáctica sobre algo que también podrías escribir a mano con eventos.

```
1  <!-- BookForm.razor -->
2  <h3>Alta de libro</h3>
3
```

```razor
<label>Título</label>
<input @bind="titulo" />

<label>ISBN</label>
<input @bind="isbn" />

<p>@mensaje</p>

@code {
    private string titulo = string.Empty;
    private string isbn = string.Empty;
    private string mensaje = string.Empty;
}
```

Los campos `titulo` e `isbn` son C# común y corriente — no tienen ningún atributo especial. Es el `@bind` del lado del marcado el que hace todo el trabajo de conectar el input con el campo.

Eventos: `@onclick` y `EventCallback`

`@onclick` conecta el evento `click` del DOM con un método C# del `@code`. A diferencia de JavaScript, el método no recibe un objeto `event` crudo salvo que lo pidas explícitamente — alcanza con declarar un método sin parámetros.

```razor
<!-- continuación de BookForm.razor -->
<button @onclick="Guardar">Guardar</button>

@code {
    // ...campos titulo, isbn, mensaje ya declarados arriba

    private void Guardar()
    {
        mensaje = $"Libro '{titulo}' (ISBN {isbn}) guardado.";
    }
}
```

Cuando el botón necesita comunicarle algo al componente padre — en vez de resolverlo puertas adentro como hace `Guardar()` — el tipo correcto para exponer no es un `Action` plano sino `EventCallback` (o `EventCallback<T>` si lleva un dato). Blazor lo integra con su ciclo de renderizado: después de

invocar un `EventCallback`, el framework sabe que tiene que volver a renderizar los componentes involucrados. Un `Action` común funcionaría para la lógica, pero perdería ese enganche automático.

Parámetros de componente: `[Parameter]`

Un componente puede recibir datos desde quien lo usa, de la misma forma que un método recibe argumentos. La propiedad se marca con el atributo `[Parameter]`, y quien instancia el componente en su propio marcado se lo pasa como si fuera un atributo HTML más.

```razor
<!-- BookSummary.razor — componente hijo, sin lógica propia -->
<div class="book-summary">
    <h4>@Titulo</h4>
    <p>ISBN: @Isbn</p>
</div>

@code {
    // [Parameter] — el valor lo asigna quien usa el componente, no él mismo
    [Parameter]
    public string Titulo { get; set; } = string.Empty;

    [Parameter]
    public string Isbn { get; set; } = string.Empty;
}
```

```razor
<!-- uso desde el componente padre — "libro" es, por ejemplo, un elemento -->
<!-- de una lista que el padre ya tiene en su propio @code -->
<BookSummary Titulo="@libro.Titulo" Isbn="@libro.Isbn" />
```

Este mecanismo es el que va a permitir, más adelante, que solo el componente **raíz** de una pantalla resuelva sus servicios por Inyección de Dependencias, y que los componentes hijos reciban lo que necesitan como parámetro — sin ir cada uno a buscar sus propias dependencias. La razón de por qué eso importa se explica en la próxima sección.

Inyección de servicios en un componente: `@inject` vs. scope

propio

Un componente puede pedir un servicio registrado en el contenedor de DI con la directiva `@inject`, igual que cualquier clase pide sus dependencias por constructor. La diferencia importante — y la razón de este anexo — es **de qué scope sale ese servicio**.

> **Un dato que cambia todo en Blazor Hybrid:** a diferencia de una app ASP.NET Core tradicional, donde cada request HTTP crea un scope de DI nuevo, **el lado cliente de Blazor no tiene concepto de scopes por request**. Un servicio registrado como `Scoped` se comporta, en la práctica, como `Singleton`: una sola instancia para toda la vida de la aplicación, salvo que algo cree un scope nuevo explícitamente. (Fuente: documentación oficial de Blazor, sección "Service lifetime".)

Eso significa que un `@inject IBookSession Session` escrito en cualquier componente siempre trae *la misma* instancia, sin importar cuántas veces el usuario abra y cierre la pantalla. Si esa instancia guarda estado de una operación puntual — como va a pasar con el ViewModel de una pantalla — ese estado queda pegado entre una apertura de pantalla y la siguiente.

La solución: `OwningComponentBase`

`OwningComponentBase` es una clase base de componente que crea su **propio scope de DI**, atado a la vida del componente: se abre cuando el componente se instancia, se descarta — junto con todo lo que se resolvió en él — cuando el componente se destruye. Expone una propiedad protegida `ScopedServices` (un `IServiceProvider`) para resolver servicios desde ese scope propio.

Para los ejemplos que siguen, este servicio mínimo alcanza para ver el efecto de cada mecanismo — guarda solo la hora en la que se creó:

```
1  public interface IBookSession
2  {
3      DateTime OpenedAt { get; }
4  }
5
6  public sealed class BookSession : IBookSession
7  {
8      // se fija una sola vez, al construirse la instancia
```

```
    public DateTime OpenedAt { get; } = DateTime.Now;
}

// registrado como Scoped — el ciclo de vida que expone el problema
services.AddScoped<IBookSession, BookSession>();
```

Con esto registrado, cada vez que algo resuelve `IBookSession` desde un scope nuevo, `OpenedAt` tiene una hora distinta — eso es lo que permite *ver* si dos resoluciones comparten scope o no.

```
<!-- BookSessionDemo.razor -->
@inject IBookSession SesionViaInject
@inherits OwningComponentBase

<p>Vía @inject (scope ambiente): @SesionViaInject.OpenedAt</p>
<p>Vía scope propio: @sesionViaScope?.OpenedAt</p>

@code {
    private IBookSession? sesionViaScope;

    protected override void OnInitialized()
    {
        // resuelto desde el scope propio del componente, no del ambiente
        sesionViaScope = ScopedServices.GetRequiredService<IBookSession>();
    }
}
```

Ojo con esta trampa: heredar de `OwningComponentBase` no alcanza por sí solo. Un servicio traído con `@inject` a secas *sigue* saliendo del scope ambiente — solo lo que se resuelve explícitamente a través de `ScopedServices` (o de la propiedad `Service`, ver abajo) usa el scope propio del componente. Por eso `SesionViaInject` y `sesionViaScope` del ejemplo de arriba, aunque se ven parecidos, no son la misma instancia.

`OwningComponentBase<TService>` — la forma corta

Cuando el componente necesita, sobre todo, **un** servicio principal con vida atada a la pantalla — el caso típico de un ViewModel — la variante genérica evita escribir `ScopedServices.GetRequiredService` a mano: expone directamente una propiedad `Service` ya resuelta desde el scope propio.

```
<!-- BookSessionCard.razor -->
@inherits OwningComponentBase<IBookSession>
```

```
3
4  <p>Sesión abierta: @Service.OpenedAt</p>
```

`Service` es de tipo `IBookSession` y sale del scope propio del componente — cuando el componente se destruye (por ejemplo, al cerrar la pantalla), ese scope y todo lo resuelto en él se descarta con él.

El proyecto que compila componentes: SDK de Razor, paquete e importaciones

Un componente Blazor es un archivo `.razor`. Para que compile y funcione, el proyecto que lo contiene necesita tres cosas, y las tres se declaran de forma explícita: nada es automático.

1. El SDK de Razor

La primera línea de un `.csproj` dice con qué kit se compila. Con el kit común (`Microsoft.NET.Sdk`) los archivos `.razor` se ignoran en silencio. Con `Microsoft.NET.Sdk.Razor` se compilan. Se obtiene creando el proyecto con la plantilla "biblioteca de clases de Razor".

2. El paquete con los tipos de Blazor

`ComponentBase`, `OwningComponentBase<T>` y los eventos de HTML viven en el paquete NuGet `Microsoft.AspNetCore.Components.Web`. Sin él la compilación falla con "el espacio de nombres AspNetCore no existe". La versión debe ser de la misma rama que el framework del proyecto (en un proyecto `net9.0`, la 9.x).

3. El archivo _Imports.razor

En un `.cs` los `using` van en cada archivo. En Blazor, los `@using` comunes a todos los componentes de una carpeta se declaran una sola vez en `_Imports.razor`. El que nunca debe faltar es `@using Microsoft.AspNetCore.Components.Web`, porque contiene la definición de los eventos (`@onclick`, `@bind`, `@onchange`…).

_Imports.razor: cómo funciona y cómo usarlo bien

En un archivo `.cs` cada archivo empieza con sus `using`. Un componente Razor también puede llevar sus `@using` arriba, pero en la práctica se repetirían en todas las Views. Para eso existe `_Imports.razor`: un archivo de texto con los `@using` que comparten los componentes. Estas son las reglas para usarlo correctamente:

- **El nombre es exacto:** `_Imports.razor` (guion bajo, *I* mayúscula, plural). Con cualquier otro nombre, el compilador lo trata como un componente más y no lo aplica.
- **Se aplica solo, sin referenciarlo:** no hay que invocarlo desde ninguna View. El compilador de Razor lo detecta y agrega esos `@using` a todos los `.razor` de *su carpeta y de las subcarpetas*.
- **Puede haber más de uno:** uno en la raíz del proyecto para lo común, y otros dentro de una carpeta para lo que solo necesitan sus componentes. Los de la raíz y los de la carpeta se acumulan.
- **Qué va ahí:** la línea de eventos web que trae la plantilla (`@using Microsoft.AspNetCore.Components.Web`) y los namespaces de los tipos que usan tus Views, como los de los ViewModels (`@using LibraryApp.InterfaceAdapter.Books.ViewModels`). **Qué no va:** lógica ni código.
- **La línea de eventos web nunca se borra:** sin ella la View compila, pero `@bind` y `@onclick` no responden.
- **Si cambia un namespace, se actualiza acá:** si movés un ViewModel a otro namespace, hay que corregir la línea correspondiente del `_Imports.razor` (en las Views no hay nada que tocar).

Cómo diagnosticar cuando una View dice que no encuentra un tipo, en este orden: (1) ¿el `@using` del namespace de ese tipo está en el `_Imports.razor`? (2) ¿el proyecto de Views referencia al proyecto que contiene ese tipo? (3) ¿está instalado el paquete NuGet que lo define? Casi siempre es la primera.

Así se ve la diferencia en una View. Sin `_Imports.razor`, cada componente tendría que repetir las líneas de importación; con él, empieza directamente por lo que lo hace único:

```
1  @using Microsoft.AspNetCore.Components.Web
2  @using LibraryApp.InterfaceAdapter.Books.ViewModels
```

Dos fallos silenciosos: con el SDK común, la compilación termina sin errores pero el componente no existe. Sin el `@using` de eventos, el componente compila y se muestra, pero `@bind` y `@onclick` quedan como atributos HTML comunes y no responden. En ninguno de los dos casos el compilador avisa: se detectan renderizando el componente y manejándolo (con una librería de pruebas de componentes como bUnit).

Errores comunes

1. Usar `@inject` para un servicio que necesita aislarse por pantalla

ERROR COMUN

El servicio queda pegado al scope ambiente — la misma instancia sobrevive a que el usuario cierre y vuelva a abrir la pantalla, arrastrando estado de la operación anterior.

```
<!-- MAL: @inject trae siempre la misma instancia en Blazor Hybrid -->
@inject IBookSession Session
```

BUENA PRACTICA

El componente raíz de la pantalla hereda de `OwningComponentBase<T>` — la instancia vive y muere con esa apertura de pantalla.

```
<!-- BIEN: scope propio, se descarta al cerrar la pantalla -->
@inherits OwningComponentBase<IBookSession>
```

2. Anidar `OwningComponentBase` en un componente hijo

ERROR COMUN

Cada componente que hereda de `OwningComponentBase` crea **su propio** scope, independiente del de cualquier otro componente. Si el padre y un hijo heredan cada uno por su cuenta, dejan de compartir la misma instancia Scoped — rompiendo la invariante de "una operación, un Presenter/UseCase".

```
<!-- MAL: BookScreen y BookSessionBadge piden el mismo tipo, -->
<!-- pero cada uno resuelve su PROPIO scope — instancias distintas -->
<!-- BookScreen.razor -->
@inherits OwningComponentBase<IBookSession>

<p>Padre: @Service.OpenedAt</p>
<BookSessionBadge />

<!-- BookSessionBadge.razor — hijo, con su propio OwningComponentBase -->
@inherits OwningComponentBase<IBookSession>

<p>Hijo: @Service.OpenedAt</p>
<!-- muestra un OpenedAt distinto al del padre: es OTRA instancia -->
```

BUENA PRACTICA

Solo el componente raíz de la pantalla resuelve por DI. Los hijos reciben lo que necesitan como `[Parameter]`, compartiendo el único scope de la pantalla.

```razor
<!-- BIEN: un solo scope, el hijo recibe datos, no resuelve servicios -->
<!-- BookScreen.razor -->
@inherits OwningComponentBase<IBookSession>

<p>Padre: @Service.OpenedAt</p>
<BookSessionBadge OpenedAt="@Service.OpenedAt" />

<!-- BookSessionBadge.razor — hijo, sin OwningComponentBase -->
<p>Hijo: @OpenedAt</p>

@code {
    [Parameter]
    public DateTime OpenedAt { get; set; }
}
```

Referencia rápida

Mecanismo	De dónde sale el servicio	Cuándo usarlo
`@inject`	Scope ambiente/raíz — en Blazor Hybrid, vive toda la app	Servicios sin estado o genuinamente `Singleton`
`OwningComponentBase` + `ScopedServices`	Scope propio del componente	Necesitás varios servicios distintos con vida = pantalla
`OwningComponentBase<TService>`	Scope propio del componente	Un solo servicio principal con vida = pantalla (típico: el ViewModel raíz)